LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 2
Student Name: K. Sirisha
Roll No.: A24126552260
Section: B

1. TITLE

Student Profile Generator Using JavaScript Class and DOM

2. AIM

To develop a Student Profile Generator web application that uses a JavaScript class to model student data and the DOM to dynamically create and display a student profile.

3. OBJECTIVE

The objectives of this experiment are:

- To define a JavaScript class representing a Student object.
- To capture form input values using the DOM.
- To validate that all required fields are filled before creating a profile.
- To dynamically generate DOM elements to display the student's profile.
- To attach an event listener to a button to trigger profile creation.
- To update the DOM in real time when the button is clicked.

4. THEORY

A JavaScript class is a template for creating objects that bundle related data (properties) and behaviour (methods) together. The constructor() method initializes an object's properties when it is created with the new keyword.

The Document Object Model (DOM) represents an HTML page as a tree of objects that JavaScript can read and modify. Methods such as document.getElementById(), document.createElement(), element.appendChild() and element.innerHTML are used to read form values and build page content dynamically, without reloading the page. Event listeners, attached using addEventListener(), let the page respond to user actions such as clicking a button.

In this application, a Student class stores a student's name, roll number, department and CGPA. When the "Create Student Profile" button is clicked, the values entered in the form are validated, a new Student object is created, and a profile card is built and inserted into the page using DOM methods.

5. ALGORITHM / PROCEDURE

1. Create a new HTML file with a form (Name, Roll Number, Department, CGPA) and a container div for the profile.
2. Define a Student class with a constructor that stores name, rollNo, department and cgpa.

3 Select the "Create Student Profile" button using document.getElementById().

4 Attach a click event listener to the button.

5 Inside the listener, read the values of all form fields using their element ids.

6 Validate that none of the fields are empty; show an alert if any field is missing.

7 Create a new Student object using the entered values.

8 Clear any previously displayed profile from the container.

9 Use document.createElement() and innerHTML to build heading and detail elements for the profile.

10 Append all the created elements to the profile card, and append the card to the container.

11 Save the file, open it in a browser, and create a student profile to verify the behaviour.

6. PROGRAM

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
    scale=1.0">
    <title>Student Profile</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <h1>Student Profile Generator</h1>
        <div class="form-box">
            <label>Name:</label>
            <input type="text" id="name" placeholder="Enter your
    name">
            <label>Roll Number:</label>
            <input type="text" id="rollNo" placeholder="Enter roll
    number">
            <label>Department:</label>
            <input type="text" id="department" placeholder="Enter
    department">
            <label>CGPA:</label>
            <input type="number" id="cgpa" step="0.01"
    placeholder="Enter CGPA">
            <button id="createProfile">Create Student
    Profile</button>
        </div>
        <div id="profileContainer"></div>
    </div>

    <script>
        // Student class
        class Student {
            constructor(name, rollNo, department, cgpa) {
                this.name = name;
                this.rollNo = rollNo;
                this.department = department;
                this.cgpa = cgpa;
            }
        }

        // Select button using DOM
        const button = document.getElementById("createProfile");
```

```
    // Event handling
    button.addEventListener("click", function () {

        // Get user-provided values
        const name = document.getElementById("name").value;
        const rollNo = document.getElementById("rollNo").value;
        const department =
document.getElementById("department").value;
        const cgpa = document.getElementById("cgpa").value;

        // Check whether all fields are entered
        if (name === "" || rollNo === "" || department === "" ||
cgpa === "") {
            alert("Please enter all student details.");
            return;
        }

        // Create Student object
        const student = new Student(
            name,
            rollNo,
            department,
            cgpa
        );

        // Select profile container
        const profileContainer =
            document.getElementById("profileContainer");

        // Clear previous profile
        profileContainer.innerHTML = "";

        // Dynamically create profile card
        const profile = document.createElement("div");
        profile.className = "profile";

        // Create heading
        const heading = document.createElement("h2");
        heading.textContent = "Student Profile";

        // Create student details
        const nameElement = document.createElement("p");
        nameElement.innerHTML =
            "<strong>Name:</strong> " + student.name;

        const rollElement = document.createElement("p");
        rollElement.innerHTML =
            "<strong>Roll No:</strong> " + student.rollNo;

        const departmentElement = document.createElement("p");
        departmentElement.innerHTML =
            "<strong>Department:</strong> " +
student.department;

        const cgpaElement = document.createElement("p");
        cgpaElement.innerHTML =
            "<strong>CGPA:</strong> " + student.cgpa;

        // Add elements to profile
        profile.appendChild(heading);
        profile.appendChild(nameElement);
        profile.appendChild(rollElement);
        profile.appendChild(departmentElement);
```

```
        profile.appendChild(cgpaElement);

        // Add profile to webpage
        profileContainer.appendChild(profile);
    });
</script>

<style>
    * {
        box-sizing: border-box;
    }
    body {
        margin: 0;
        font-family: Arial, sans-serif;
        background: #f2f4f7;
    }
    .container {
        width: 500px;
        margin: 50px auto;
        text-align: center;
    }
    h1 {
        color: #333;
    }
    .form-box {
        background: white;
        padding: 25px;
        border-radius: 10px;
        box-shadow: 0 4px 10px rgba(0, 0, 0, 0.15);
        text-align: left;
    }
    label {
        display: block;
        margin-top: 12px;
        margin-bottom: 5px;
        font-weight: bold;
    }
    input {
        width: 100%;
        padding: 10px;
        border: 1px solid #aaa;
        border-radius: 5px;
    }
    button {
        width: 100%;
        margin-top: 20px;
        padding: 12px;
        border: none;
        border-radius: 5px;
        background: #007bff;
        color: white;
        font-size: 16px;
        cursor: pointer;
    }
    button:hover {
        background: #0056b3;
    }
    .profile {
        margin-top: 25px;
        padding: 25px;
        background: white;
        border-radius: 10px;
        box-shadow: 0 4px 10px rgba(0, 0, 0, 0.2);
```

```
            text-align: left;
        }
        .profile h2 {
            text-align: center;
            margin-bottom: 20px;
        }
        .profile p {
            font-size: 17px;
            margin: 12px 0;
        }
    </style>
</body>
</html>
```

7. CODE EXPLANATION

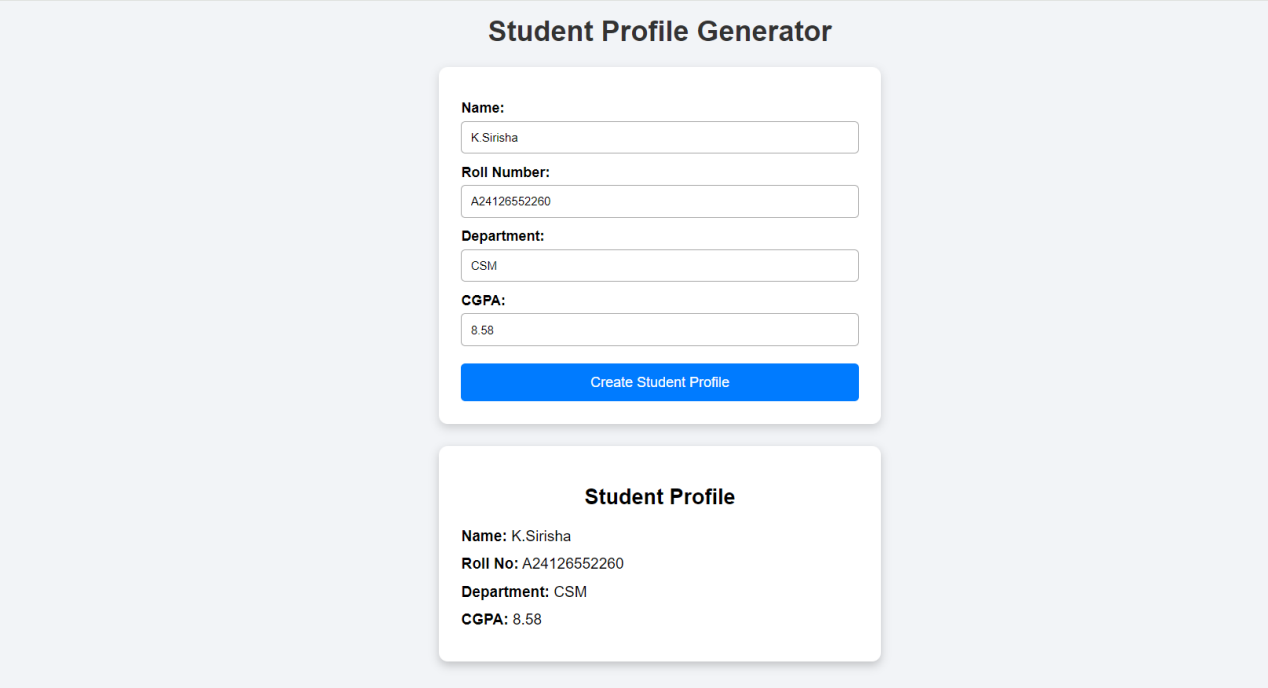
Code Explanation

Code	Explanation
class Student { constructor(...) }	Defines the blueprint for a single student object with name, rollNo, department and cgpa.
document.getElementById("createProfile")	Selects the "Create Student Profile" button from the DOM.
button.addEventListener("click", ...)	Listens for a click on the button to trigger profile creation.
.value	Reads the current value entered in each form input field.
if (... === "") { alert(...); return; }	Validates that all fields are filled before creating a profile.
new Student(name, rollNo, department, cgpa)	Creates a new Student object using the entered form values.
document.createElement("div") / ("h2") / ("p")	Creates new DOM elements to represent the profile card and its details.
element.innerHTML	Sets the formatted text (with bold labels) inside a profile detail element.
appendChild()	Attaches a created element as a child of another element to build the profile card.
profileContainer.innerHTML = ""	Clears any previously displayed profile before showing the new one.

8. TEST CASES AND EXECUTION DATA

Test Case	**Action / Input**	**Expected Result**	**Actual Result**	**Status**
TC-01	Load the page in browser	Form is displayed with empty fields and no profile shown	Displayed correctly	Pass
TC-02	Click "Create Student Profile" with all fields empty	An alert asks to enter all student details	Alert displayed as expected	Pass
TC-03	Fill Name, Roll Number, Department and CGPA and click the button	A profile card is created showing the entered details	Profile card displayed correctly	Pass
TC-04	Create a profile, then change values and click the button again	The previous profile is cleared and the new one is shown	Old profile replaced correctly	Pass
TC-05	Enter a decimal value in the CGPA field	The CGPA field accepts decimal numbers	Decimal value accepted	Pass

9. OUTPUT



10. RESULT

Thus, a Student Profile Generator web application was successfully developed using a JavaScript class to model student data and DOM manipulation to dynamically create and display a student's profile. The output was verified against the expected results for all test cases.